PROGRAMAÇÃO DE BOLSO RFO BASIC

AULA

01



O celular que fala

Voz, variáveis e decisões, direto do bolso.

RECURSO DO CELULAR

Síntese de voz (TTS)

variáveis · texto · decimais · entrada de dados · condições

Projeto de Ensino em Programação · Novembro de 2026

NESTA AULA VOCÊ VAI

- ✓ Fazer o aparelho **falar** e escrever na tela.
- ✓ Guardar informações em **variáveis** (números e textos).
- ✓ Perguntar o nome do usuário e reagir à resposta com um `if`.

Programar costuma começar com uma tela preta e um “Olá, mundo” mudo. Aqui o “Olá, mundo” **fala**. Em poucos minutos o seu celular vai cumprimentar você pelo nome, calcular sua idade e comentar o resultado. Tudo isso sem computador: você escreve, executa e testa no mesmo aparelho.

CONCEITO · ALGORITMO

Um programa é uma lista de instruções executadas uma após a outra, de cima para baixo. O celular segue a receita sem interpretar nada, então cada instrução precisa ser exata.

1 Seu primeiro programa

Abra o aplicativo **hBasic**. No canto superior esquerdo deve haver o nome **unnamed program** que significa que o programa não possui um nome, pois não foi salvo ainda.

Na primeira linha deve estar escrito **REM Start of BASIC! Program**, isto é, início do programa. A palavra-chave **REM** (Remark) significa que esta linha é um comentário, uma observação, e será ignorada quando executarmos o programa. Fique à vontade para apagá-la, se desejar.

DICA

Para iniciar um novo programa, selecione a opção **Clear** no menu.

Em seguida, digite o código abaixo no editor:

Listagem 1: Saudação

`hello.bas` `basic`

```
1  tts.init
2  tts.speak "Olá! Eu sou o seu celular."
3  tts.stop
4  print "Olá! Eu sou o seu celular."
5  end "Fim do Programa."
```

SALVAR Salve o programa. Escolha um nome, `hello`, por exemplo!

No canto superior direito aparecerá `hello.bas`. Antes de executar, vá ao menu e selecione **Format**. O texto que você criou será formatado para o padrão BASIC, onde as palavras-chave são capitalizadas e as linhas são indentadas apropriadamente.

Esta linguagem não é *case sensitive*, ou seja, tanto faz escrever `print` quanto `Print` ou `PRINT`, por exemplo.

RODAR Aumente o volume! Abra o menu e escolha **Run**.

```
Olá! Eu sou o seu celular.
Fim do Programa.
```

Cada linha é uma instrução. Veja o que cada uma faz:

Comando	O que faz
`tts.init`	Liga o sintetizador de voz. Deve vir antes de qualquer fala.
`tts.speak`	Fala o texto que vem depois, entre aspas.
`tts.stop`	interrompe a reprodução atual
`print`	Imprime o texto entre aspas na tela.
`end`	Indica que o programa finalizou. É opcional

ATENÇÃO

A sigla TTS significa Text to Speech (Texto para Fala). O texto que o celular fala ou escreve vai sempre **entre aspas**. As aspas dizem: “isto é um texto, não um comando”. Ademais, o comando `end` pode ser usado em qualquer lugar do código. Ele imprime a mensagem e para a execução do programa. Se nenhum texto for passado para `end`, será escrito na tela “END”.

2 Variáveis

Escrever a mesma frase várias vezes é cansativo. Para guardar uma informação e reutilizá-la, usamos uma **variável**.

CONCEITO · VARIÁVEL

Uma variável é uma caixinha com uma etiqueta. A etiqueta é o **nome** e o conteúdo é o **valor**.

O hBasic tem dois tipos básicos de variável, distinguidos pelo nome:

Tipo	Como se escreve	Exemplo
Texto	nome **com** `$` no final	`nome$ = "Ana"`
Número	nome sem `$`	`idade = 17`

Um programa que usa variáveis de texto:

Listagem 2: Saudação com variável

variavel.bas basic

```
1  nome$ = "Ana"
2  mensagem$ = "Olá, " + nome$ + "! Bem-vinda à programação de bolso."
3  tts.init
4  tts.speak mensagem$
5  tts.stop
6  print mensagem$
7  end
```

,

RODAR Execute e depois troque `"Ana"` pelo seu nome.

```
Olá, Ana! Bem-vinda à programação de bolso.
```

ATENÇÃO

O sinal `=` significa “guarde isto aí”, e não “é igual a” como na matemática. Primeiro declaramos a variável com um rótulo seguido do sinal de igualdade e o valor a ser armazenado.

RFO Basic também usa o comando `let` para denotar uma operação de atribuição, mas pode ser opcional. Se você escrever uma variável cujo o nome inicia com uma palavra-chave de BASIC!, obterá um erro. Portanto, você deverá usar `let` neste caso. Existem palavras reservadas da linguagem e elas não podem aparecer no ínicio de uma linha. Por exemplo, `printer` é visto como `print er`. Para usar `printer` como variável escreva `let printer`.

DICA

Escolha nomes que contem uma história: `idade` e `nome$` dizem mais do que `x` e `a$`. Você vai agradecer quando reler o programa daqui a um mês.

Juntando textos

Um número em RFO Basic é sempre um decimal. Isso significa que quando imprimimos, por exemplo 17, o resultado na tela será 17.0.

Listagem 3: Concatenando texto com número

variavel.bas basic

```
1  idade = 17
2  print idade
3  print "Idade: " + str$(idade) + " anos."
```

O sinal `+` entre textos os **concatena**, ou seja, cola um no outro. Para colar um **número** num texto, é preciso convertê-lo antes com `str$()`. Troque `str$` por `int$` e veja a diferença.

> **ATENÇÃO**
>
> `"Idade: " + idade` provoca erro, porque não se pode misturar texto e número sem converter.

Você deve ter notado que o comando `print` escreve um texto na tela do seu celular e salta para a próxima linha. Se o texto que quisermos mostrar for muito grande, podemos usar `;` para evitar a quebra de linha.

Listagem 4: Impressão sem nova linha

`variavel.bas` `basic`

```
1 print "Idade: ";
2 print idade
```

3 Perguntando ao usuário

Um programa fica interessante quando conversa. O comando `input` mostra uma pergunta e guarda a resposta numa variável:

Listagem 5: Entrada de dados

`input.bas` `basic`

```
1  input "Como você se chama?", nome$
2  input "Em que ano você nasceu?", nasc
3
4  idade = 2026 - nasc
5  frase$ = nome$ + ", você tem cerca de " + int$(idade) + " anos."
6  tts.init
7  tts.speak frase$
8  tts.stop
9  print frase$
10
11 end
```

,

Note que `nome$` recebe texto e `nasc` recebe um número. A linha `idade = 2026 - nasc` mostra que variáveis também fazem contas.

RODAR Execute com seus dados.

4 Tomando decisões

Até agora o programa seguiu sempre o mesmo caminho. Com `if`, ele escolhe caminhos diferentes conforme uma **condição**:

Símbolo	Significa	Exemplo
`=`	igual a (numa condição)	`idade = 18`
`<>`	diferente de	`idade <> 18`
`<`	menor que	`idade < 18`
`>`	maior que	`idade > 18`
`<=`	menor ou igual a	`idade <= 18`
`>=`	maior ou igual a	`idade >= 18`

Juntando tudo, temos o **assistente de bolso**:

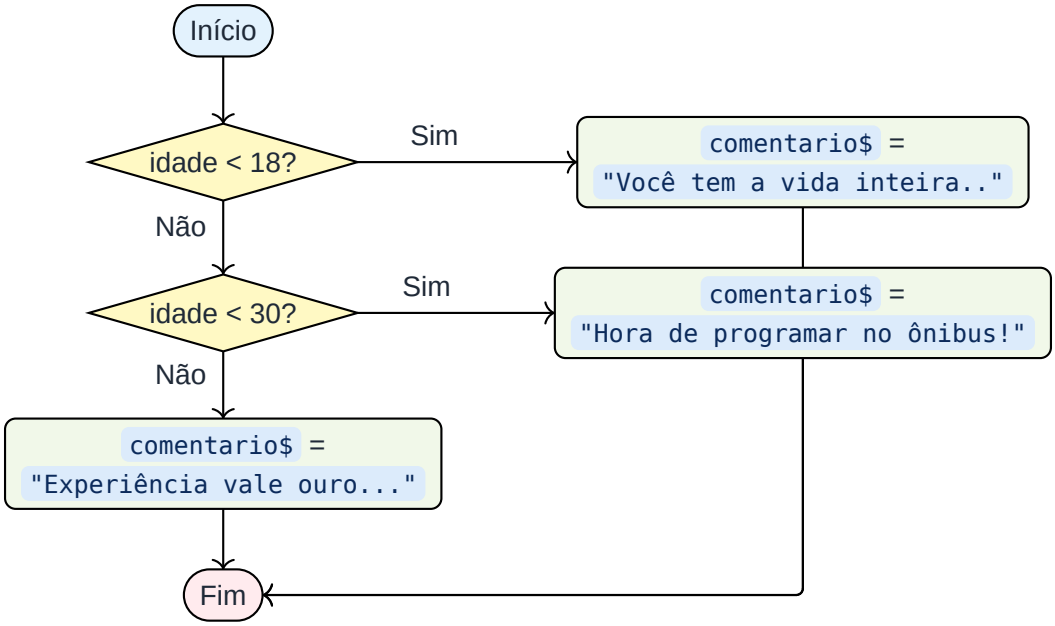
Listagem 6: Entrada de dados com decisão

`input-if.bas` `basic`

```
1   input "Como você se chama?", nome$
2   input "Em que ano você nasceu?", nasc
3   idade = 2026 - nasc
4   saudacao$ = "Olá, " + nome$ + "! Você tem " + int$(idade) + " anos."
5
6   if idade < 18
7     comentario$ = "Você tem a vida inteira pela frente."
8   elseif idade < 30
9     comentario$ = "Hora de programar no ônibus!"
10  else
11    comentario$ = "Experiência vale ouro na programação."
12  endif
13
14  tts.init
15  tts.speak saudacao$
16  tts.speak comentario$
17  tts.stop
18
19  print saudacao$
20  print comentario$
```

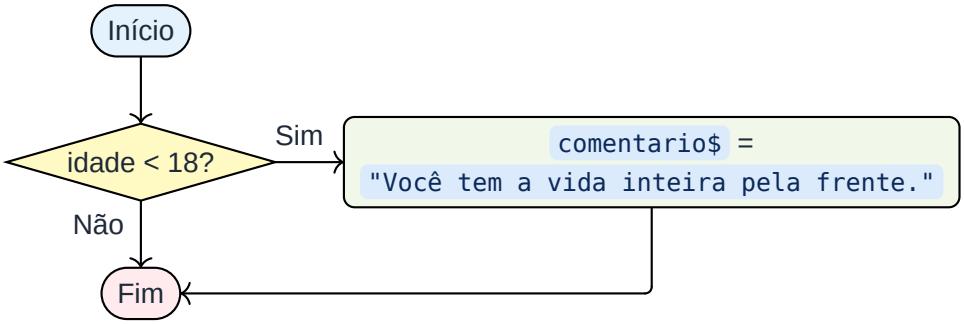
CONCEITO · CONDIÇÃO

O `if` testa uma pergunta de resposta **sim ou não**. Se for sim, executa o bloco abaixo dele. Se for não, passa para o `elseif` seguinte e, por fim, para o `else`. Só **um** dos blocos é executado, e o `endif` marca onde a decisão termina.



Se o seu bloco condicional contiver apenas uma instrução, você pode usar uma forma mais simples, toda em uma única linha. Nesta forma, a palavra-chave `then` é obrigatória, e não existem `elseIf` nem `endif`.

```
if idade < 18 then comentario$ = "Você tem a vida inteira pela frente."
```



ATENÇÃO

O sinal de igualdade é usado com dois sentidos. Primeiro, quando queremos atribuir um valor a uma variável:

- `nota = 7.5`
- `conceito$ = "B"`

Esse tipo de operação não é uma equação como aprendemos na escola. Por exemplo, `nota = nota + 1` significa: “pegue o valor atual de `nota`, some 1 e guarde o resultado de volta em `nota`”, ou seja, `nota` agora é 8.5. O outro caso que usamos igualdade é em algum `if`, por exemplo:

- `if nota = 10`: nota é igual a 10?
- `elseif nota = 8: nota é igual a 8?`

Nesse caso, o sinal de igualdade funciona como um teste de uma condição e não como uma atribuição de valor. O valor de `nota` não muda.

5 Quebre o código!

Programadores aprendem muito lendo mensagens de erro. Faça cada experimento abaixo no assistente de bolso, **um de cada vez**, e anote o que acontece:

QUEBRAR Apague a linha `tts.init`. O que o celular faz?

QUEBRAR Troque `nome$` por `nome` em uma das linhas. Aparece algum erro? qual?

QUEBRAR Tire o `str$( )` ao redor de `idade`. O que o interpretador reclama?

QUEBRAR Apague o `endif`. Aparece algum erro? Use datas que caiam nas três situações

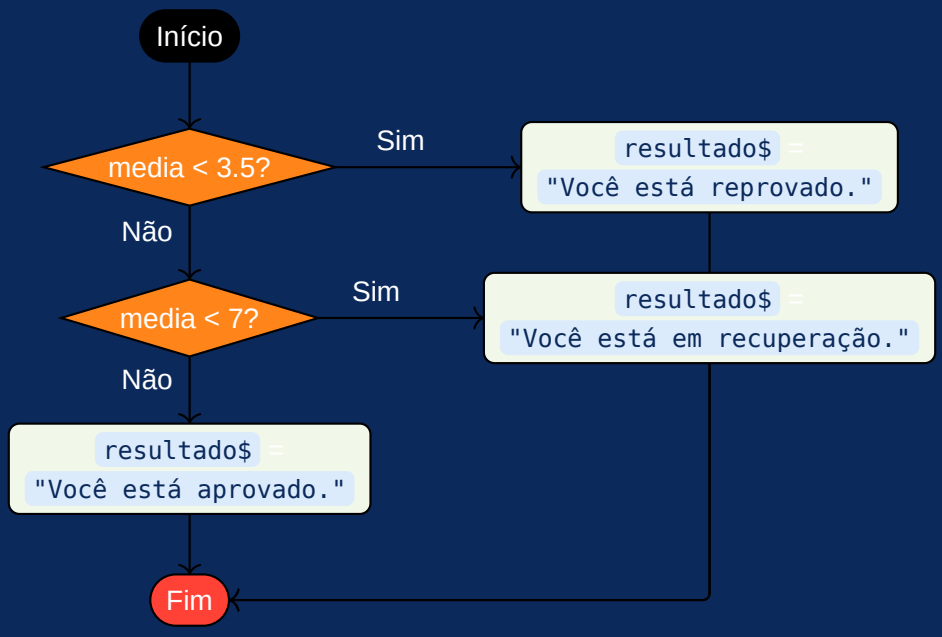
QUEBRAR Mude `2026 - nasc` para `nasc - 2026`. O programa acusa erro? A resposta faz sentido?

> **DICA**
>
> O celular obedece a receita **exatamente como escrita**, mesmo quando o resultado é absurdo. Um erro de lógica não gera mensagem de erro, e quem precisa perceber é você.

DESAFIO Assistente falante

Crie um programa novo que:

1. pergunte a nota da primeira, segunda e terceira avaliação;
2. calcule a média aritmética das três notas;
3. use um `if` para determinar se o aluno foi aprovado;
4. escreva e faça o celular falar o resultado.



6 Resumo

Comando	Para que serve
`tts.init`	Liga a voz do celular.
`tts.speak "..."`	Fala o texto.
`print "..."`	Escreve na tela.
`input "?", v$`	Pergunta e guarda a resposta na variável.
`nome$ = "..."`	Cria uma variável de **texto**.
`idade = 17`	Cria uma variável **numérica**.
`if / elseif / else / endif`	Escolhe caminhos conforme uma condição.

NA PRÓXIMA AULA

Aula 2: Expressões. Vamos escrever expressões matemáticas e lógicas. Vamos aprofundar nossos conhecimentos do `if`, conhecer o gerador de números aleatórios e como abrir uma caixa de diálogo.